Concurrent Task Dispatcher

Design Report

Randy Cantu | CSCI 3334-01 | Systems Programming | Spring 2026

1. Overview

This project is a concurrent task dispatcher written in Rust. Tasks arrive over time, enter a shared ready queue, and are routed by a dedicated dispatcher thread to a bounded pool of four worker threads that execute them concurrently. The system demonstrates ownership, threads, channels, shared state, scheduling, metrics, and clean shutdown using only Rust's standard library.

Two scheduling policies were implemented and compared: a baseline FIFO policy with round-robin worker assignment, and an optimized Shortest Job First (SJF) approximation that prefers shorter tasks from a look-ahead window. Both were tested under a balanced workload (moderate arrival rate) and a stressed workload (fast arrivals). The FIFO vs. optimized comparison is the primary experiment.

2. Architecture, Components, and Data Structures

The pipeline has five stages:

```
Generator (main thread)
   |  Arc<Mutex<VecDeque<Task>>>
   Ready Queue
   |
   Dispatcher Thread  [DispatchPolicy: Fifo | OptimizedSjfApprox]
   |  mpsc channel per worker
   Worker Threads x4
   |  Arc<Mutex<Metrics>>
   Shared Metrics
```

The generator (main thread) creates 500 tasks and enqueues them one at a time with randomized sleep intervals, so tasks arrive over real time. The dispatcher runs in its own thread, pulling tasks from the queue according to the active policy and routing each one to the next worker via round-robin. Four worker threads each own the receiver end of an mpsc channel, simulate execution with thread::sleep(duration_ms), and update shared metrics on completion.

Key data structures: VecDeque<Task> gives O(1) push_back and pop_front for FIFO ordering (and push_front for reinserting candidates under the optimized policy). The queue is wrapped in Arc<Mutex<...>> for safe shared access between the generator and dispatcher. Worker assignment uses per-worker mpsc channels, which means workers never contend over a shared lock to receive tasks. Metrics are collected in an Arc<Mutex<Metrics>> struct updated by all workers. A DispatchPolicy enum (Fifo or OptimizedSjfApprox) is passed to the dispatcher at startup so policy selection does not require touching any other module.

3. Synchronization Strategy

Two strategies handle concurrency in this system. For shared mutable state (the ready queue and the metrics struct), Arc<Mutex<T>> is used. Arc provides shared ownership across threads without violating Rust's borrowing rules. Mutex ensures only one thread reads or writes at a time, preventing data races. The lock is held only briefly in both cases, just long enough to push, pop, or update a value, which keeps contention low.

For task delivery from the dispatcher to workers, mpsc channels are used instead of another shared mutex. Once the dispatcher selects a task, it sends it through the target worker's channel. The worker receives it exclusively, with no further locking needed. Channels also handle shutdown automatically: when the dispatcher drops all Sender handles at the end of a run, workers detect the closed channel through their while let Ok(task) = rx.recv() loop and exit cleanly. No explicit shutdown flags, condition variables, or poison pills are needed.

4. Task Model and Scheduling Policies

Each Task struct contains an id (u32), an arrival_time (u64), a kind (TaskKind: Cpu or Io), a duration_ms (u64), and a created_at Instant recorded at construction. The created_at field is what allows accurate wait time and turnaround time calculations later. Tasks are generated with a fixed seed (42) for reproducibility, with durations randomly drawn from 50 to 200 ms and a roughly 50/50 CPU/IO split. Both task kinds enter the same queue and are treated identically by both policies.

FIFO (Baseline): the dispatcher calls pop_front and sends whatever is at the head of the queue. No task is ever skipped. Worker assignment cycles through the four senders in round-robin order. This is simple, predictable, and starvation-free, but long tasks at the front block shorter ones behind them (the convoy effect).

Optimized SJF Approximation: instead of taking the front task blindly, the dispatcher locks the queue, pulls up to 8 tasks into a local buffer, finds the one with the smallest duration_ms, dispatches it, and pushes the remaining tasks back to the front in their original order. LOOK_AHEAD = 8 limits how aggressively long tasks are bypassed, which is a deliberate fairness tradeoff. Worker assignment still uses round-robin. Under light load the queue is shallow and the scan overhead is not justified. Under heavy load with a deep queue, the policy has real candidates to evaluate and meaningfully reduces max wait time and makespan.

5. Scheduling Policy Comparison (Primary Experiment)

Both policies were run on both workloads using the same seeded task set. Output files are named by policy and workload: fifo_balanced_output.txt, optimized_stressed_output.txt, etc.

Balanced Workload (arrival gap 5 to 30 ms)

Metric	FIFO	Optimized SJF
Makespan	15,734 ms	15,844 ms
Avg Wait Time	7,839.13 ms	7,857.86 ms
Avg Turnaround	7,962.65 ms	7,981.69 ms
Max Wait Time	15,555 ms	15,663 ms
CPU / IO Tasks	259 / 241	259 / 241

Stressed Workload (arrival gap 1 to 5 ms)

Metric	FIFO	Optimized SJF
Makespan	16,153 ms	15,905 ms

Metric	FIFO	Optimized SJF
Avg Wait Time	7,898.05 ms	7,896.82 ms
Avg Turnaround	8,022.78 ms	8,021.79 ms
Max Wait Time	16,081 ms	15,771 ms
CPU / IO Tasks	247 / 253	247 / 253

Interpretation

The results show that the benefit of the optimized policy depends entirely on workload conditions. Under the balanced workload, FIFO came out slightly ahead across all metrics. With arrival gaps of 5 to 30 ms, the queue stays relatively shallow most of the time, so the SJF window rarely has meaningful candidates. The overhead of scanning up to 8 tasks, removing one, and reinserting the rest adds more cost than the smarter selection saves.

Under the stressed workload, the picture reverses. With tasks arriving every 1 to 5 ms, the queue stays deep throughout the simulation. The SJF window consistently has varied candidates to evaluate, and the policy can avoid dispatching long tasks when shorter ones are already waiting. Makespan dropped by 248 ms and max wait time dropped by 310 ms compared to FIFO. The key takeaway is that scheduling improvements are context-dependent: FIFO is the better choice under light load, and the SJF approximation earns its overhead only when the queue stays persistently deep.

Comparing the two workloads also confirms that worker capacity is the dominant performance constraint. Average wait times barely changed between balanced and stressed runs under either policy, because the drain rate is fixed by the four workers regardless of how fast tasks arrive.

6. Metrics Collected

All metrics are computed from real timestamps recorded during task processing. Required metrics: total tasks completed, makespan (wall-clock time from first task to last completion), average wait time (arrival to execution start), and average turnaround time (arrival to full completion). Additional metrics: max wait time, CPU task count, and IO task count. Results are printed to the terminal and saved automatically to a labeled .txt file at the end of each run.

7. Bugs, Shutdown, and Lessons Learned

Three bugs came up during development. Module import errors caused by using mod task; in files other than main.rs were fixed by consolidating module declarations in main.rs and using use crate::task::Task; everywhere else. A Task struct mismatch (different field names across files) was fixed by making task.rs the single source of truth for the struct definition. The most significant bug was workers running infinite loop {} blocks with no exit path, which caused the program to hang after all tasks were dispatched. Replacing the infinite loop with while let Ok(task) = rx.recv() fixed it: the loop exits automatically when the dispatcher drops all Sender handles, giving the system clean shutdown with no flags or extra signals.

The most practical lessons: building the system in layers (task, queue, generator, workers, dispatcher, metrics, in that order) made each component verifiable before wiring it to the next. Adding the second scheduling policy only required changing dispatcher.rs because the architecture kept each concern isolated. And measuring both workloads under both policies was essential, the data showed the optimized policy does not always win, which would not have been obvious without running the comparison.

8. Trade-offs and Conclusion

Known limitations: the worker pool is fixed and does not scale with load. CPU and IO tasks share the same queue under both policies, so type-aware routing is not currently possible. Round-robin assignment does not account for current worker load. Under the optimized policy, long tasks can be repeatedly deferred if shorter tasks keep arriving in the window, which is a fairness concern moderated by the LOOK_AHEAD cap of 8.

The primary finding is that scheduling policy performance is workload-dependent: FIFO is slightly faster under balanced load due to lower overhead, and the SJF approximation reduces makespan and max wait time under stressed load where queue depth gives it real selection opportunities. The most impactful next improvement would be shortest-queue dispatch to replace round-robin, followed by separate CPU and IO queues with weighted scheduling between them.